

Tarea 1 — Utilitarios de administración en Bash

CC5308 Administración de Sistemas Linux

Profesor: Ismael Figueroa

Descripción

Esta tarea consiste en implementar **cuatro scripts *utilitarios* de automatización de línea de comandos en Bash**, utilizando lo visto en clases: navegación y manipulación del filesystem, redirección de I/O, pipes y filtros, y expansiones con *quoting*.

La tarea es **individual** y se entrega un único archivo (**.tar.gz** o **.zip**) que se evalúa de forma **automática** con un script de corrección.

Requisitos previos

- Contar con un entorno **Ubuntu 24.04**: puede usar el servidor del curso.
- No se requiere instalar paquetes adicionales ni conexión a Internet: todo se resuelve con el sistema base (**bash**, **coreutils**, **grep**, **tar**, etc.).

Integridad académica

La tarea es individual. No está permitido compartir soluciones ni alimentar asistentes (ChatGPT u otros) con los enunciados. Sí puedes usarlos para consultas técnicas puntuales. Cualquier falta se rige por los procedimientos de la Escuela/Facultad.

Consideraciones

1. Cada utilitario es un **script Bash** ejecutable (con *shebang* **#!/usr/bin/env bash**).
2. Solo se pueden usar herramientas estándar del sistema (**grep**, **cut**, **sort**, **uniq**, **wc**, **head**, **tail**, **tar**, etc.) y elementos de Bash como iteraciones, condicionales u otros.
3. **No** se permite usar lenguajes interpretados o compilados externos (Python, Perl, Ruby, Node, C, Go, etc.) ni descargar herramientas.
4. Los scripts deben mostrar su forma de uso al recibir el argumento **-h** o **--help**, y al recibir una cantidad incorrecta de argumentos.
5. Los scripts deben ser robustos: si reciben argumentos inválidos (archivo o directorio inexistente, cantidad de argumentos incorrecta), deben terminar con un código de salida distinto de cero y un mensaje de error en **stderr**.
6. Los nombres de archivo deben coincidir de manera exacta con lo solicitado: **analizar_log.sh**, **organizar.sh**, **reporte.sh** y **backup.sh**.

Entorno de evaluación

Utilice el comando **make check** para evaluar su entrega automáticamente. El script de evaluación (**check.sh**) genera sus propios datos de prueba y ejecuta cada script contra casos deterministas, comparando salidas y estado del filesystem. Además de los casos iniciales presentes en la tarea, su entrega se evaluará sobre casos ocultos.

1. `analizar_log.sh` — resumen de un log de acceso web

Uso: `./analizar_log.sh <archivo_log>`

Recibe la ruta de un archivo de log de un servidor web en **NCSA Common Log Format**, donde cada línea tiene la forma:

```
192.168.1.10 - - [20/Oct/2026:09:00:01 -0300] "GET /index.html HTTP/1.1" 200 5234
```

Esto es: `<ip> - - [fecha] "GET <ruta> HTTP/1.1" <estado> <bytes>`.

Imprime en stdout, **en este orden y con este formato exacto**, el resumen:

Total de peticiones: `<N>`

Peticiones con error: `<M>`

IP con más peticiones: `<IP> (<K> peticiones)`

Direcciones IP únicas: `<U>`

donde:

- `<N>` — total de líneas del log.
- `<M>` — líneas cuyo **código de estado** es un error de cliente o servidor (códigos 4xx o 5xx). El estado es el campo que viene después de la petición entre comillas (pista: `cut` con delimitador `"`).
- `<IP>` y `<K>` — la dirección IP que más aparece en el log y cuántas veces.
- `<U>` — cantidad de direcciones IP distintas.

Ejemplo (con `data/access.log`):

Total de peticiones: 30

Peticiones con error: 7

IP con más peticiones: 192.168.1.10 (6 peticiones)

Direcciones IP únicas: 8

Hint: utilice pipes y filtros (`cut`, `grep`, `sort`, `uniq`, `wc`, `head`), here-strings, sustitución de comandos. Utilice `data/access.log` para probar su script.

2. `organizar.sh` — agrupar archivos por extensión

Uso: `./organizar.sh <directorio>`

Dado un directorio, mueve cada **archivo regular** (sin recursión) a un subdirectorio del propio directorio cuyo nombre es su **extensión en minúsculas** (sin el punto). Reglas:

- La extensión es la parte **después del último punto** del nombre (`archivo.tar.gz` → `gz/`). Si el nombre no tiene punto, va a `sin_extension/`.
- La extensión se **normaliza a minúsculas** (`Informe.TXT` → `txt/`).
- Los **archivos ocultos** (cuyo nombre empieza con `.`) se ignoran por completo.
- Los **subdirectorios** y otros tipos de archivo se ignoran.
- Los subdirectorios de destino se crean según sea necesario (`mkdir -p`).

Al finalizar imprime en `stdout`:

Archivos organizados: <N>
Categorías: <cat1> <cat2> ...

donde <N> es la cantidad de archivos movidos y <cat1> <cat2> ... la lista de extensiones utilizadas, **ordenadas alfabéticamente** y separadas por un espacio.

Hint: `usemkdir -p`, `mv`, `globbing ("${dir}/*")`, pruebas `[ -f ]`, expansión de parámetros `${var##*.}`, `loops for`.

3. `reporte.sh` — reporte de un directorio

Uso: `./reporte.sh <directorio>`

Analiza el directorio dado (sus archivos regulares directos, sin recursión ni ocultos) y **escribe** un archivo `reporte.txt` en el **directorio de trabajo actual** con el siguiente formato exacto:

```
=====
REPORTE DE DIRECTORIO
-----
Generado por   : <usuario>
Fecha y hora   : <fecha y hora>
Directorio     : <ruta del directorio>
-----
Archivos totales      : <N>
Archivos vacíos       : <V>
Tamaño total (bytes)  : <B>
Tamaño promedio      : <P> bytes por archivo
=====
```

donde:

- <usuario> — la variable de entorno `USER` (o `whoami`).
- <fecha y hora> — la fecha y hora actual (ej. `date '+%Y-%m-%d %H:%M:%S'`).
- <N> — cantidad de archivos regulares.
- <V> — cuántos de ellos están vacíos (tamaño 0).
- — suma de los tamaños en bytes (ej. `wc -c`).
- <P> — promedio entero / <N> (si no hay archivos, 0).

Al terminar imprime en `stdout`:

Reporte generado en `reporte.txt`

Hint: redirección `>`, here-document `<<`, variables de entorno `$USER`, sustitución de comandos `$(...)`, aritmética `$(...)`.

4. `backup.sh` — respaldo comprimido de un directorio

Uso: `./backup.sh <directorio> [directorio_destino]`

Crea un archivo `tar.gz` con el contenido del directorio indicado. El destino es `directorio_destino` si se entrega; si no, el directorio de trabajo actual (se crea si no existe). El nombre del archivo es:

`backup_<nombre_base>_<marca_de_tiempo>.tar.gz`

donde `<nombre_base>` es el nombre del directorio (`basename`) y `<marca_de_tiempo>` es `date '+%Y%m%d_%H%M%S'`. El respaldo debe **preservar la estructura interna** del directorio (incluyendo subdirectorios) y funcionar correctamente con **rutas y nombres que contienen espacios**.

Al terminar imprime en `stdout`:

Respaldo creado: `<ruta_absoluta_del_archivo>`

Hint: utilice `tar`, sustitución de comandos `$(date ...)`, expansión de parámetros `$(basename ...)`, *quoting* para espacios.

Entrega

Entrega **un único archivo** por U-Cursos:

- `tarea1_<login>.tar.gz` o `tarea1_<login>.zip`

El archivo debe contener:

Archivo	Obligatorio
<code>analizar_log.sh</code>	sí
<code>organizar.sh</code>	sí
<code>reporte.sh</code>	sí
<code>backup.sh</code>	sí
<code>README.md</code>	sí

El `README.md` debe explicar brevemente cómo ejecutar cada utilitario y cualquier decisión de diseño relevante (cómo extraer el estado del log, cómo nombras los respaldos, cómo maneja `organizar.sh` los casos borde, etc.).

Autoevaluación

Se puede probar la implementación con el evaluador local antes de subirla. Desde la carpeta que contiene `check.sh`:

```
./check.sh ruta/a/tu/entrega.tar.gz      # o .zip, o una carpeta descomprimida
```

El evaluador imprime el detalle por ítem y el puntaje final (0–100). También puedes probar cada script a mano:

```
./analizar_log.sh data/access.log
mkdir -p /tmp/prueba && touch /tmp/prueba/nota.txt /tmp/prueba/LEEME.md /tmp/prueba/sin_ext
./organizar.sh /tmp/prueba
./reporte.sh /tmp/prueba && cat reporte.txt
./backup.sh /tmp/prueba
```

Evaluación

La evaluación consiste en:

- Evaluación automática en los casos publicados en la tarea.
- Evaluación automática en casos ocultos.
- Evaluación manual por parte del equipo docente.

Los puntajes se distribuyen de la siguiente forma:

Sección	Puntos
A. Formalidades (scripts presentes, shebang, ejecutables, README)	10
B. Script <code>analizar_log.sh</code>	25
C. Script <code>organizar.sh</code>	25
D. Script <code>reporte.sh</code>	20
E. Script <code>backup.sh</code>	20
Total	100

Exigencia 60 %: **60 puntos = nota 4.0.**